

Universidad de Costa Rica
Facultad de Ingeniería
Escuela Ciencias de la Computación e Informática
Programación Paralela y Concurrente

**Proyecto Semestral de Programación Paralela
y Concurrente (II entregable)**

CI-0117

Grupo 01

Profesor:

Sleyter Angulo

Elaborado por:

Sasha Chavarría Arias C32078

Justin Steve Burgos Muñoz C31361

Emanuel Hernández Arauz B93699

2026-06-09

Tabla de Contenidos

1. Introducción	2
2. Objetivos	2
3. Descripción del Problema	2
4. Estructura	2
4.1 Diagrama de flujo	2
4.2 Diseño del programa	2
4.2.1 Pthreads	2
4.2.2 MPI	
4.3 Módulos - Funciones	3
5. Manual del Usuario	3
5.1 Requerimientos de Hardware	3
5.2 Requerimientos de Software	4
5.3 Compilación	4
5.4 Ejecución	4
5.5 Pruebas - Resultados	4
6. Referencias o Bibliografía	5

1. Introducción

El presente proyecto implementa una solución paralela para el alineamiento exacto de secuencias de ADN utilizando la búsqueda de múltiples patrones. Mediante un enfoque de fuerza bruta, se acelera la identificación de cadenas empleando Pthreads para la ejecución en memoria compartida y MPI para entornos de memoria distribuida.

2. Objetivos

Implementar un algoritmo de fuerza bruta capaz de buscar múltiples patrones en secuencias largas de ADN.

Acelerar el tiempo de búsqueda dividiendo la carga de trabajo mediante las tecnologías de paralelismo Pthreads y MPI.

Manejar adecuadamente las condiciones de carrera y la sincronización de datos mediante exclusión mutua y operaciones colectivas.

3. Descripción del Problema

El ácido desoxirribonucleico (ADN) es una organización dentro de las células de cada ser vivo, formados por material genético, siendo el gen la unidad física y funcional de la herencia. El factor hereditario muestra desde rasgos generales como lo pueden ser color de ojos hasta enfermedades como la diabetes.

“No siempre es fácil determinar si una afección en una familia es hereditaria. Un especialista en genética puede revisar los antecedentes familiares de salud de una persona (un registro de información médica sobre su familia inmediata y extendida).” (National Library of Medicine, 2021)

Por lo tanto, esta es la importancia del material genético, mostrando en los conjuntos de genes la clave para determinar información relevante del sujeto de estudio. Estos conjuntos de genes mantienen su estructura gracias al apareamiento de las bases nitrogenadas, que permiten la estructura característica del ADN (Clínica Universidad de Navarra, 2023). Las bases nitrogenadas se dividen entre Purinas y Pirimidinas; las Purinas poseen una estructura de doble anillo (adenina y guanina), mientras que las Pirimidinas una estructura de un solo anillo (citosina y timina). Cada una destacando por su inicial A, G, C y T.

En este proyecto, se requiere identificar la primera posición exacta en la que ocurren K patrones de nucleótidos dentro de una secuencia principal de ADN de longitud N . Al procesar secuencias de gran tamaño (ej. millones de caracteres), la búsqueda puramente secuencial resulta ineficiente, por lo que es indispensable paralelizar la comparación.

4. Estructura

4.1 Diagrama de flujo

A continuación se muestra un diagrama del proyecto general y diagramas individuales de cada parte solicitada en el proyecto, las cuales corresponden a la solución serial, paralela e interfaz de paso de mensajes (MPI):

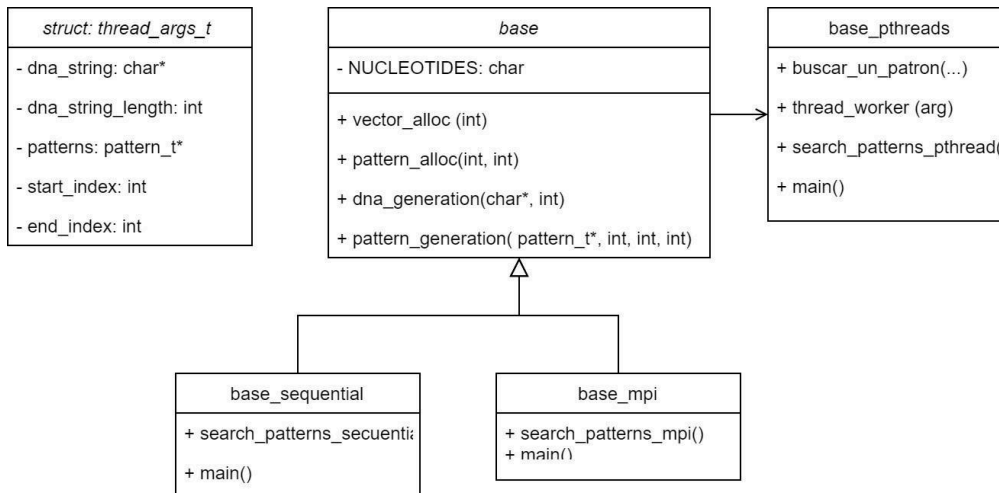


Figura 1. Diagrama estructural y procedural del proyecto, según la implementación creada.

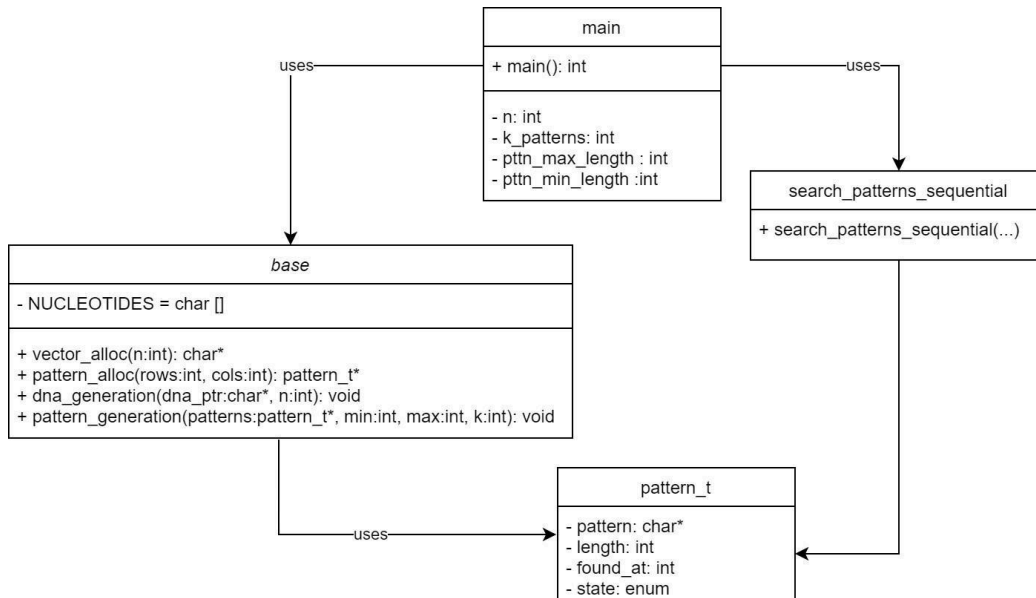


Figura 2. Diagrama de funciones en el programa secuencial según base_sequential.

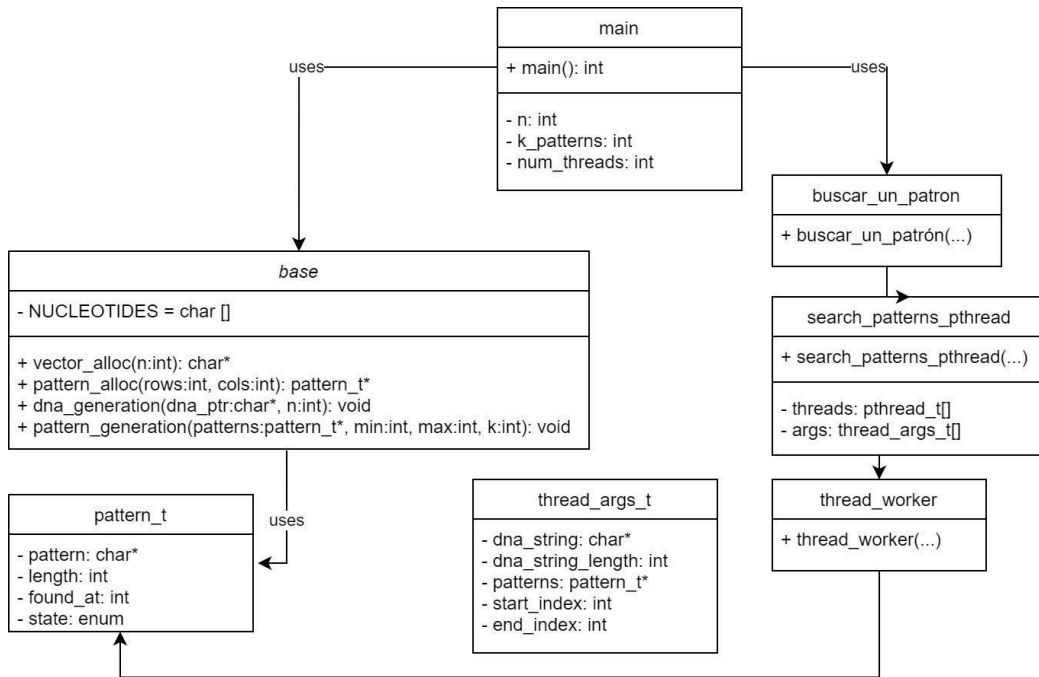


Figura 3. Diagrama de funciones en el programa con hilos según base_pthreads.

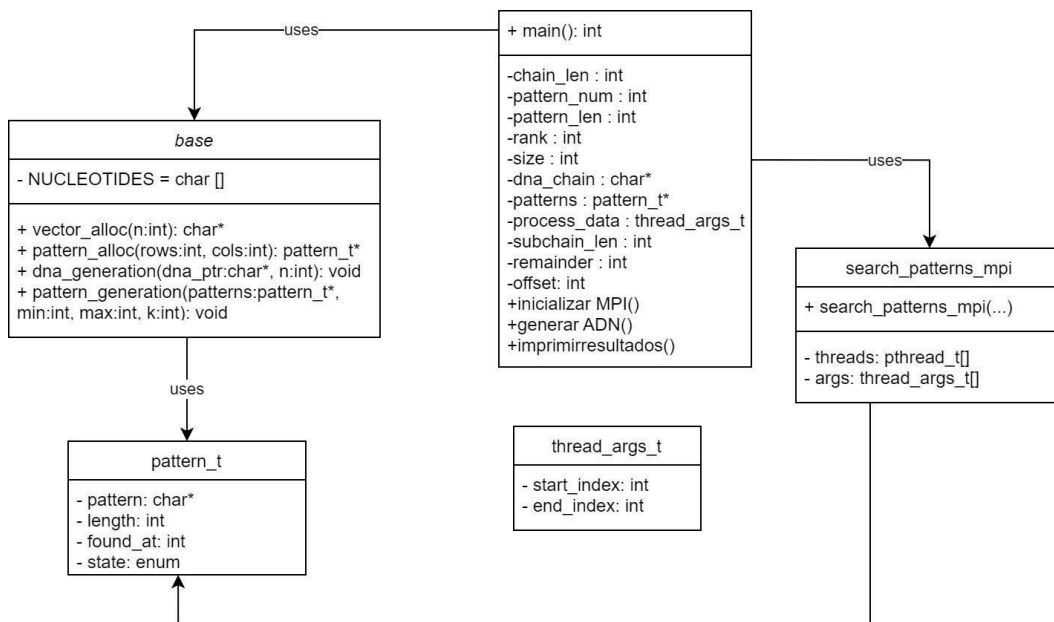


Figura 4. Diagrama de funciones en el programa MPI según base_mpi.

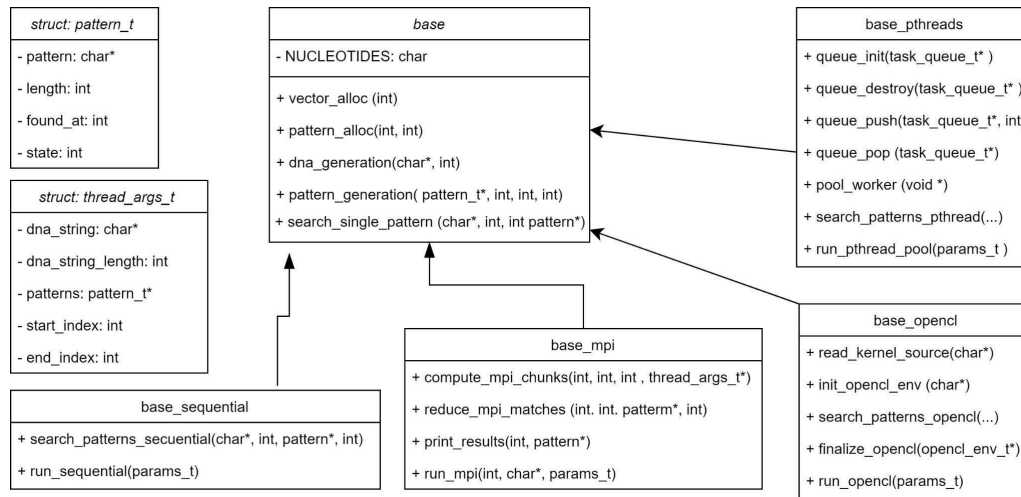


Figura 5. Diagrama estructural del proyecto en la fase 2.

4.2 Diseño del programa

4.2.1 Pthreads

La estrategia de procesamiento en Pthreads se basa en la división equitativa de la carga de trabajo: la cantidad total de patrones a evaluar (K), se distribuye de manera uniforme entre la totalidad de los hilos de ejecución instanciados. Más concretamente, cada hilo recibe un bloque de patrones de tamaño K/N , donde N es el número total de hilos. En caso de existir un número K impar, un único hilo realizará una sobrecarga o descarga sobre los demás, donde el trabajo asignado para este será de $(K/N) - 1$ o $(K/N) + 1$ patrones.

Una vez asignado su bloque de patrones, cada hilo opera de manera completamente independiente. La evaluación de los patrones asignados por cada hilo se realiza sobre una misma y única secuencia principal de datos de entrada, la cual actúa como un recurso compartido de solo lectura para todos los procesos concurrentes. Esta aproximación garantiza que la evaluación de un subconjunto de patrones no requiere ninguna forma de comunicación o sincronización con los demás hilos durante la fase de cálculo, ya que todos acceden a la secuencia de entrada de forma no modificable. Esta independencia en el procesamiento es fundamental para maximizar el paralelismo y reducir la sobrecarga de sincronización, permitiendo que la búsqueda de cada patrón dentro de su respectivo bloque se ejecute simultáneamente.

Por consiguiente, la estructura anteriormente mencionada fue editada buscando su optimización con tres aplicaciones nuevas; crear y destruir hilos en cada búsqueda, manteniendo un conjunto de hilos siempre activos por medio de mutex. Asignar patrones a hilos para evitar que algunos hilos terminen antes. Y aplicar `pthread_cond_wait` y `pthread_cond_signal` para coordinar hilos. Esto fue aplicado por medio de una cola de tareas que es bloqueada con mutex para mantener el orden de las tareas, de ahí, comienza a utilizar las condiciones para esperar elementos disponibles o avisar si contiene espacio libre.

4.2.2 MPI

La estrategia de procesamiento en MPI emplea un modelo de memoria distribuida liderado por un proceso maestro (rango 0), encargado de la generación inicial de los datos. A

diferencia de la implementación con Pthreads, la carga de trabajo aquí se divide segmentando la secuencia principal de ADN en lugar de dividir el arreglo de patrones. Primero, el proceso maestro transmite una copia idéntica de la secuencia completa y de todos los patrones a los demás nodos participantes mediante operaciones de comunicación colectiva (MPI_Bcast).

Una vez que cada nodo tiene los datos en su memoria local, la longitud total de la secuencia de ADN se divide equitativamente entre el número de procesos. Cada proceso calcula un índice de inicio y un índice de fin, asumiendo la responsabilidad de un segmento específico de la cadena (incluyendo el manejo de residuos si la división no es exacta). Todos los procesos evalúan la totalidad de los K patrones, pero restringen su ciclo de búsqueda estrictamente al fragmento de la secuencia que les fue asignado. Esta partición espacial del entorno de búsqueda asegura que los nodos trabajen de forma concurrente y aislada, sin necesidad de comunicarse ni sincronizarse durante la fase de cálculo de fuerza bruta.

Al finalizar la exploración de su respectivo segmento, cada proceso genera un arreglo de resultados locales que indica el estado (encontrado o no) y la posición de cada uno de los K patrones. Estos arreglos independientes son recolectados por el proceso maestro mediante la operación MPI_Gather. Finalmente, el maestro ejecuta una fase de consolidación de manera secuencial, dado que un mismo patrón pudo haber sido hallado por múltiples procesos en diferentes partes de la secuencia, el maestro itera sobre los resultados recopilados y preserva únicamente la coincidencia (MATCH) que posea el índice global más bajo, garantizando así que se reporte la primera aparición absoluta del patrón en la cadena original.

4.2.3 OpenCL

Esta implementación consta de una reestructuración interna dada por un estándar abierto que toma en cuenta la arquitectura de la máquina utilizada para aprovechar al máximo sus recursos, tomando en cuenta el CPU, hardware, GPU, etc.

Su funcionamiento fue aplicado de la siguiente forma; obtiene la plataforma y dispositivo en uso por medio de clGetPlatformIDs y clCreateContext. A continuación, lee un archivo tipo kernel (que posee la estructura característica de división de núcleos) y a partir de esto, corre la estructura original de búsqueda de patrones de ADN asegurando la eficiencia y utilidad máxima de la arquitectura y logrando los objetivos del código.

4.3 Módulos -

Funciones base.h

vector_alloc, pattern_alloc: Asignación de memoria dinámica para secuencias y estructuras.

dna_generation, pattern_generation: Llenado pseudoaleatorio de secuencias usando los nucleótidos A, T, C, G.

base_sec.h

search_patterns_sequential: Implementación base de búsqueda unihilo.

base_pth.h

search_patterns_pthread: Creación de hilos y asignación de rangos de búsqueda.

base_mpi.h

search_pattern_mpi: Ejecución del cálculo en nodos distribuidos y reducción de métricas.

base_opencl.h

search_pattern_openc1: Implementación de estructura con aprovechamiento en GPU.

5. Manual del Usuario

5.1 Requerimientos de Hardware

No presenta requerimientos mínimos (Interfaz CLI), pero se recomienda su ejecución en procesadores de al menos 2 núcleos.

5.2 Requerimientos de Software

Sistema Operativo: Ubuntu 24.04 LTS (Distribución base).

Herramientas: Compilador GCC, OpenMPI, biblioteca POSIX Threads.

5.3 Compilación

El proyecto se provee de un Makefile, encargado de facilitar al usuario la compilación del mismo.

Secuencial: `make sec`

`gcc -O2 -o dna_sec base_sequential.c`

base.c Pthreads: `make pthread`

`gcc -O2 -o dna_pthread base_pthread.c base.c`

-lpthread MPI: `make mpi`

`mpicc -O2 -o dna_mpi base_mpi.c base.c`

NOTA: Para limpieza de archivos generados use `make clean`.

5.4 Ejecución

El programa permite la inyección de parámetros para definir la longitud de la secuencia de ADN (-n) y la cantidad de patrones a buscar (-k).

Ejecución Secuencial:

```
./dna_sec -n 1000000 -k 100
```

Ejecución Pthreads:

```
./dna_pthread -n 1000000 -k 100
```

Ejecución con MPI (4 procesos):

```
mpirun -np 4 ./dna_mpi -n 1000000 -k 100
```

NOTA: Los procesos están ligados directamente con el número de hilos máximo del equipo en el que se está trabajando.

5.5 Pruebas - Resultados

```
justbm@justinbm-Lenovo: ~/Desktop/Alineamient...
make: *** [Makefile:50: test] Error 1
justbm@justinbm-Lenovo:~/Desktop/Alineamiento-Secuencias-ADN$ make test
mpicc -Wall -Wextra -Wpedantic -std=c11 -Iinclude -DCL_TARGET_OPENCL_VERSION=300
-c src/base_openc1.c -o build/base_openc1.o
mpicc build/main.o build/base.o build/base_sequential.o build/base_pthreads.o bu
ild/base_mpi.o build/base_openc1.o build/params.o build/tests.o -lpthread -lopen
CL -o bin/dna_search
Ejecutable creado en ./bin/dna_search
Ejecutando suite de pruebas unitarias...
./bin/dna_search -m 5
Ejecutando con: n=50000, k=10, l=3, t=4, m=5
=== EJECUTANDO PRUEBAS UNITARIAS (TESTS) ===

PASS Caso 1 Secuencial
PASS Caso 1 Pthread
PASS Caso 1 OpenCL
PASS Caso 2 Secuencial
PASS Caso 2 Pthread
PASS Caso 2 OpenCL
PASS Caso 3 Secuencial
PASS Caso 3 Pthread
PASS Caso 3 OpenCL
PASS Caso 4 Secuencial
PASS Caso 4 Pthread
PASS Caso 4 OpenCL
PASS Caso 5 Comparacion (Secuencial == Pthreads == OpenCL)

Pruebas pasadas: 13/13
justbm@justinbm-Lenovo:~/Desktop/Alineamiento-Secuencias-ADN$
```

Versión	Tamaño (N)	Patrones (K)	Longitud (L)	Hilos	Tiempo Real	Estado
Secuencial	100,000,000	50	3	1	1.970s	Completado
Pthreads (Pool)	100,000,000	50	3	4	2.696s	Completado
OpenCL	100,000,000	50	3	N/A	2.284s	Fallido (Segmentation fault)

Versión	Tamaño (N)	Patrones (K)	Longitud (L)	Hilos	Tiempo Real	Speedup vs Secuencial
Secuencial	10,000,000	50	15	1	3.97s	1.00x (Ref)
Pthreads (Pool)	10,000,000	50	15	4	1.91s	~2.08x
OpenCL	10,000,000	50	15	N/A	1.24s	~3.20x

Al incrementar la dificultad del procesamiento (evaluando patrones de 15 nucleótidos en lugar de 3, lo que exige una exploración más exhaustiva de la cadena sin hallar resultados prontos), las implementaciones de alto desempeño exhiben su capacidad real de aceleración.

La arquitectura mejorada con Pthreads permitió disminuir la latencia a la mitad (1.91s), minimizando el impacto del costo de instanciación mediante la técnica de *Pool*. Asimismo, el estándar OpenCL registró la mayor optimización (1.24s), superando por más de tres veces el desempeño del programa base. Estos resultados validan que tratar cada segmento de la hélice genética como un *work-item* autónomo constituye un enfoque sumamente eficiente ante volúmenes de datos masivos.

6. Referencias o Bibliografía

ChatGPT. (2026). Funciones de condiciones en Pthreads. OpenAI.
<https://chatgpt.com/c/6a27a18c-04b4-83e8-80ce-80c69fb8b3a2>

Clínica Universidad de Navarra (2023) *Base nitrogenada: definición médica* | *Diccionario Médico de la Clínica Universitaria de Navarra.*
<https://www.cun.es/diccionario-medico/terminos/base-nitrogenada>

Disposición de los tetrominós (2014) *How Many Colored Tetrominoes?* Disponible en:
<http://mrburkemath.blogspot.com/2014/09/how-many-colored-tetrominoes.html>
(accesado: 6 septiembre 2023).

National Library of Medicine. (2021). *¿Qué significa que un trastorno parezca ser hereditario en la familia?*. En *MedlinePlus en español.*
<https://medlineplus.gov/spanish/genetica/entender/herencia/eshereditariodefamilia/>

Wiki, C. to F. (2021) *Tetris Crazy, Fantendo Wiki.* Disponible en: https://fantendo.fandom.com/es/wiki/Tetris_Crazy (accesado: 6 septiembre 2023).